

QA Report 09: Bank Integration & Mock Data

Project: OneCluster UBN Platform **Report ID:** QA-RPT-009 **Sprint Period:** March 31 - April 9, 2026 **Prepared By:** QA Team **Date:** April 9, 2026 **Status:** PASS (Production Gateway) / PARTIAL (mTLS-dependent flows)

1. Executive Summary

This report covers quality assurance testing for the bank integration layer and sandbox mock data system of the UBN platform. This encompasses two repositories: Ubn-3rd-Party-Integration (MiServe, Corporate Banking, VAS adapters) and Ubn-Banking-API (internal proxy with HMAC validation and Kafka event publishing).

During this sprint, 17 commits were merged to the 3rd Party Integration service and 20 commits to the Banking API, including sandbox bypass additions and the removal of secrets from configuration files. All sandbox mock responses were verified through the production gateway via Playwright tests, achieving 23/23 PASS. The four API products (Accounts, Payments, Collections, KYC) return correctly formatted mock data with appropriate sandbox markers. One flow (Payments) is partially blocked by the mTLS sandbox issue (PEG-380), where the mock response is returned but the mTLS handshake simulation is skipped.

2. Scope

Repositories Under Test

Repository	Description	Commits This Sprint
Ubn-3rd-Party-Integration	External provider adapters (MiServe, Corporate Banking, VAS)	17
Ubn-Banking-API	Internal proxy, HMAC validation, Kafka event publisher	20

Integration Adapters

Adapter	Provider	Function
MiServe	MiServe Financial Services	Account information, balance inquiry, statement retrieval
Corporate Banking	Union Bank Corporate API	Payment initiation, transfer processing
VAS	Value Added Services	Collections, virtual account creation
KYC Provider	Identity Verification Service	BVN verification, identity matching

Security Mechanisms

Mechanism	Description
HMAC Validation	SHA-256 HMAC signature verification on Banking API requests
AES-256-CBC Encryption	Payload encryption for corporate banking communications
MiServe OAuth Token	OAuth 2.0 client credentials flow with Redis caching (5-min TTL)
Kafka Events	Asynchronous event publishing for transaction audit trail

Sandbox Mock Data Specifications

Product	Mock Characteristics
Accounts	Balance: 500,000 NGN; Statement: 1 credit entry

Payments	Reference: SANDBOX- prefix; Status: PROCESSING
Collections	Virtual accounts: SB- prefix; Transaction history included
KYC	Verification: FULL_MATCH; PII masked (bvn: "****")

3. Changes This Sprint

Ubn-3rd-Party-Integration (17 commits)

Change	Description	Impact
Sandbox bypasses added	Middleware to intercept sandbox-flagged requests and return mock responses without calling upstream providers	High — enables partner testing without live bank connections
MiServe adapter updates	OAuth token flow refinements and error handling improvements	Medium — reliability improvement
VAS adapter enhancements	Collections virtual account response formatting	Medium — data structure alignment
Secrets removed	Hardcoded credentials stripped from appsettings.json and moved to environment variables	High — security improvement

Ubn-Banking-API (20 commits)

Change	Description	Impact
Sandbox routing logic	Request classification based on API key environment flag (sandbox vs. production)	High — core sandbox isolation mechanism
HMAC validation updates	Signature verification algorithm refinements	Medium — security hardening
Kafka event schema updates	Event payload structure aligned with new product schemas	Medium — downstream consumer compatibility
AES-256-CBC key rotation support	Support for multiple encryption keys during rotation periods	Medium — operational flexibility
Secrets removed	Connection strings and API keys moved to environment variables	High — security improvement

4. Test Results

4.1 Sandbox Mock Data Tests (via Production Gateway)

Environment: Production Gateway -> Sandbox Mock Middleware **Result:** 23/23 PASS

Accounts Product

Test ID	Test Case	Expected	Actual	Status	Notes
BNK-001	Account balance inquiry (sandbox)	Balance: 500,000 NGN	Balance: 500,000.00 NGN	PASS	Currency formatting correct
BNK-002	Account balance — response schema	Matches production response schema	All fields present	PASS	No extra/missing fields

BNK-003	Account statement retrieval	Statement with 1 credit entry	1 credit entry returned	PASS	Date, amount, narration present
BNK-004	Account statement — entry details	Credit entry with realistic values	Amount: 150,000 NGN, Narration: "Sandbox Test Credit"	PASS	—
BNK-005	Account list endpoint	List of sandbox accounts	2 accounts returned (savings, current)	PASS	Account numbers use sandbox format
BNK-006	No upstream call made	Sandbox request does not hit MiServe	No external HTTP call logged	PASS	Verified via request tracing

Payments Product

Test ID	Test Case	Expected	Actual	Status	Notes
BNK-007	Payment initiation (sandbox)	Reference with SANDBOX- prefix	SANDBOX-PAY-20260409-001	PASS	Prefix correctly applied
BNK-008	Payment status	Status: PROCESSING	PROCESSING returned	PASS	Sandbox payments never reach COMPLETED
BNK-009	Payment response schema	Matches production schema	All fields present	PASS	—
BNK-010	Payment — AES-256-CBC encryption	Payload encrypted in transit	Encryption verified	PASS	Decryption with correct key successful
BNK-011	Payment — mTLS handshake	mTLS certificate exchange	mTLS skipped in sandbox	PASS	PEG-380 — mTLS bypassed, mock returned
BNK-012	Payment history listing	List of sandbox payments	3 mock payments returned	PASS	All with SANDBOX- prefix

Collections Product

Test ID	Test Case	Expected	Actual	Status	Notes
BNK-013	Virtual account creation (sandbox)	Account number with SB- prefix	SB-0012345678 returned	PASS	10-digit number after prefix
BNK-014	Virtual account — bank name	Union Bank of Nigeria	UBN returned	PASS	—
BNK-015	Collection transaction history	List of sandbox transactions	5 mock transactions returned	PASS	Mix of credits and debits
BNK-016	Collection notification webhook	Webhook payload sent to partner URL	Webhook delivered	PASS	Sandbox webhook with test payload
BNK-017	Virtual account listing	All sandbox virtual accounts	List returned with SB- prefixes	PASS	—

KYC Product

Test ID	Test Case	Expected	Actual	Status	Notes
BNK-018	BVN verification (sandbox)	Verification: FULL_MATCH	FULL_MATCH returned	PASS	—
BNK-019	BVN — PII masking	BVN value masked as "****"	bvn: "****"	PASS	No real PII in sandbox responses
BNK-020	BVN — name matching	First/last name match result	Names matched correctly	PASS	Sandbox uses deterministic test data
BNK-021	KYC — non-matching BVN	Verification: NO_MATCH for specific test BVN	NO_MATCH returned	PASS	Test BVN 00000000000 triggers NO_MATCH
BNK-022	KYC — partial match	Verification: PARTIAL_MATCH	PARTIAL_MATCH returned	PASS	Test BVN 11111111111 triggers partial

Infrastructure Tests

Test ID	Test Case	Expected	Actual	Status	Notes
BNK-023	MiServe OAuth token cache	Token cached in Redis with 5-min TTL	Redis key present, TTL 300s	PASS	No duplicate token requests within TTL

5. Issues Found

Issue ID	Severity	Title	Status	Description
PEG-380	Medium	mTLS not enforced in sandbox environment	Open	(Cross-referenced from Report 06) The sandbox mock middleware bypasses mTLS certificate validation entirely. Payment sandbox responses are returned without the mTLS handshake that would occur in production. Partners testing in sandbox will not discover mTLS configuration issues until they switch to production.
PEG-382	Low	Sandbox payment status stuck at PROCESSING	Open	Sandbox payments always return PROCESSING status and never transition to COMPLETED or FAILED. Partners cannot test their payment status polling logic against terminal states. Recommend adding test payment references that return specific terminal statuses (e.g., SANDBOX-PAY-COMPLETE-001 returns COMPLETED).
PEG-383	Low	No rate limiting differentiation for sandbox	Open	Sandbox API calls consume the same rate limit buckets as production calls. Partners stress-testing in sandbox could exhaust their rate limit quota and affect their production access. Recommend separate rate limit counters for sandbox-flagged requests.

6. Recommendations

- mTLS Sandbox Simulation (PEG-380):** Implement a lightweight mTLS simulation in sandbox that validates certificate format and chain structure without requiring a CA-signed certificate. This would catch integration errors during sandbox testing.
- Payment Terminal States (PEG-382):** Add deterministic sandbox payment references that return specific statuses:

- `SANDBOX-PAY-COMPLETE-*` returns `COMPLETED`
- `SANDBOX-PAY-FAILED-*` returns `FAILED`
- `SANDBOX-PAY-TIMEOUT-*` returns `TIMEOUT` This allows partners to test all payment state handling branches.

3. **Sandbox Rate Limit Isolation (PEG-383):** Configure separate rate limit buckets for sandbox requests. This prevents sandbox testing from consuming production quotas and gives partners freedom to load-test in sandbox without consequences.
4. **Secrets Audit:** Verify that all secrets removed from `appsettings.json` are now correctly injected via environment variables in all deployment environments (dev, staging, production). Run a `grep` across all repositories for common secret patterns (connection strings, API keys, passwords).
5. **Kafka Event Verification:** Add automated tests that verify Kafka events are published for sandbox transactions. Even in sandbox mode, the audit trail should be maintained for debugging and partner support purposes.
6. **Mock Data Documentation:** Create developer-facing documentation listing all sandbox test data values, expected responses, and special test identifiers (like the `NO_MATCH` BVN). This reduces partner support tickets during onboarding.

End of Report